

移动应用开发实验报告

封面信息

项目	内容
实验名称	原生应用开发
实验编号	d20301035102、d20301009202
学号	2023210651
姓名	周乐
班级	移动应用开发班
日期	2026年3月21日
组别	第6组
项目名称	智慧校园生活服务平台·智能温控系统
技术栈	.NET MAUI (C# / XAML)

实验目的

- 掌握MAUI框架下登录页面的XAML布局设计
- 理解C#后台代码与XAML界面的数据绑定机制
- 学会MAUI页面间导航与参数传递技术
- 完成用户身份验证逻辑与输入校验
- 掌握温控系统核心功能模块的实现方法

实验环境

平台	开发工具	语言	SDK版本
Windows	Visual Studio 2022	C# 12.0	.NET 9.0
Android	Visual Studio 2022	C# 12.0	API 34
iOS	Visual Studio 2022	C# 12.0	iOS 17

实验步骤

1. 需求分析

1.1 功能需求

- ☑ 用户名输入（支持学号/工号）
- ☑ 密码输入
- ☑ 登录按钮点击事件
- ☑ 输入验证（不能为空、长度限制、格式校验）
- ☑ 本地用户身份验证
- ☑ 温度显示与调节控制
- ☑ 设备开关控制（空调、地暖）
- ☑ 温度历史记录展示

1.2 界面需求

- ☑ 现代化卡片式布局设计
- ☑ Material Design风格配色
- ☑ 输入框占位提示文字
- ☑ 密码隐藏显示切换
- ☑ 温度滑块拖拽调节
- ☑ 开关状态实时显示

1.3 跳转需求

- ☑ 登录成功 → 温控系统主页
- ☑ 传递用户名与登录状态数据
- ☑ 异常处理与错误提示

2. MAUI登录页面开发

2.1 项目创建

- ☑ 打开Visual Studio 2022
- ☑ 创建.NET MAUI App项目
- ☑ 项目名称：helloworld
- ☑ 框架版本：.NET 9.0

2.2 布局实现（XAML）

```
<!-- MainPage.xaml -->
<?xml version="1.0" encoding="utf-8" ?>
<ContentPage xmlns="http://schemas.microsoft.com/dotnet/2021/maui"
    xmlns:x="http://schemas.microsoft.com/winfx/2009/xaml"
    x:Class="helloworld.MainPage"
    Title="温控系统登录"
    BackgroundColor="#F0F4F8">

    <ScrollView>
        <VerticalStackLayout Padding="40" Spacing="20" VerticalOptions="Center">

            <!-- 系统Logo -->
            <Image Source="thermostat_icon.svg"
                HeightRequest="120"
                WidthRequest="120"
                HorizontalOptions="Center" />

            <!-- 系统标题 -->
            <Label Text="智能温控系统"
                FontSize="28"
                FontAttributes="Bold"
                HorizontalOptions="Center"
                TextColor="#2C3E50" />

            <!-- 登录表单 -->
            <Frame BackgroundColor="White"
                CornerRadius="10"
                Padding="20"
                HasShadow="True">
                <VerticalStackLayout Spacing="15">

                    <!-- 用户名输入 -->
                    <Entry x:Name="UsernameEntry"
                        Placeholder="请输入用户名"
                        Keyboard="Text"
                        BackgroundColor="#F8F9FA"
                        TextColor="#2C3E50" />

                    <!-- 密码输入 -->
                    <Entry x:Name="PasswordEntry"
                        Placeholder="请输入密码"
                        IsPassword="True"
```

```
                BackgroundColor="#F8F9FA"
                TextColor="#2C3E50" />

        <!-- 登录按钮 -->
        <Button Text="登 录"
                BackgroundColor="#3498DB"
                TextColor="White"
                FontAttributes="Bold"
                CornerRadius="5"
                HeightRequest="45"
                Clicked="OnLoginClicked" />

    </VerticalStackLayout>
</Frame>

</VerticalStackLayout>
</ScrollView>

</ContentPage>
```

2.3 登录逻辑实现 (C#)

```
// MainPage.xaml.cs
namespace helloworld;

public partial class MainPage : ContentPage
{
    // 测试账号字典
    private Dictionary<string, string> testUsers = new Dictionary<string, string>
    {
        { "admin", "123456" },
        { "user", "123456" },
        { "2023210651", "654321" }
    };

    public MainPage()
    {
        InitializeComponent();
    }

    private async void OnLoginClicked(object sender, EventArgs e)
    {
        string username = UsernameEntry.Text?.Trim() ?? string.Empty;
        string password = PasswordEntry.Text?.Trim() ?? string.Empty;

        // 输入验证
        if (!ValidateInput(username, password))
        {
            return;
        }

        // 用户验证
        if (ValidateUser(username, password))
        {
            await DisplayAlert("登录成功", "欢迎使用智能温控系统", "确定");

            // 跳转到系统主页, 传递用户名
            await Navigation.PushAsync(new HomePage(username));
        }
        else
        {
            await DisplayAlert("登录失败", "用户名或密码错误", "确定");
        }
    }
}
```

```

private bool ValidateInput(string username, string password)
{
    if (string.IsNullOrEmpty(username))
    {
        DisplayAlert("提示", "用户名不能为空", "确定");
        UsernameEntry.Focus();
        return false;
    }

    if (string.IsNullOrEmpty(password))
    {
        DisplayAlert("提示", "密码不能为空", "确定");
        PasswordEntry.Focus();
        return false;
    }

    return true;
}

private bool ValidateUser(string username, string password)
{
    if (testUsers.ContainsKey(username))
    {
        return testUsers[username] == password;
    }
    return false;
}

private async void OnForgotPasswordTapped(object sender, EventArgs e)
{
    await DisplayAlert("忘记密码", "请联系管理员重置密码", "确定");
}
}

```

2.4 温控系统主页实现

```
// HomePage.xaml.cs
namespace helloworld;

public partial class HomePage : ContentPage
{
    private int _targetTemp = 25;
    private string _currentUsername;
    private ObservableCollection<TempRecord> _tempHistory = new();

    public HomePage(string username)
    {
        InitializeComponent();
        _currentUsername = username;
        WelcomeLabel.Text = $"欢迎您, {username}";

        TemperatureHistoryListView.ItemsSource = _tempHistory;

        // 模拟温度数据更新
        Device.StartTimer(TimeSpan.FromSeconds(3), () =>
        {
            UpdateTemperatureHistory();
            return true;
        });
    }

    private void OnTempSliderChanged(object sender, ValueChangedEventArgs e)
    {
        _targetTemp = (int)Math.Round(e.NewValue);
        TargetTempLabel.Text = $" {_targetTemp}°C";
    }

    private void OnAcToggled(object sender, ToggledEventArgs e)
    {
        AcStatusLabel.Text = e.Value ? "运行中" : "已关闭";
        AcStatusLabel.TextColor = e.Value ? Color.FromArgb("#27AE60") :
Color.FromArgb("#95A5A6");

        // 空调与地暖互锁
        if (e.Value)
        {
            HeaterSwitch.IsToggled = false;
        }
    }
}
```

```

    }

    private void OnHeaterToggled(object sender, ToggledEventArgs e)
    {
        HeaterStatusLabel.Text = e.Value ? "运行中" : "已关闭";
        HeaterStatusLabel.TextColor = e.Value ? Color.FromArgb("#E74C3C") :
Color.FromArgb("#95A5A6");

        // 地暖与空调互锁
        if (e.Value)
        {
            AcSwitch.IsToggled = false;
        }
    }

    private void UpdateTemperatureHistory()
    {
        MainThread.BeginInvokeOnMainThread(() =>
        {
            var random = new Random();
            var currentTemp = 20 + random.NextDouble() * 10;

            _tempHistory.Insert(0, new TempRecord
            {
                Time = DateTime.Now.ToString("HH:mm:ss"),
                Temperature = $"{currentTemp:F1}°C"
            });

            // 最多保留10条记录
            if (_tempHistory.Count > 10)
            {
                _tempHistory.RemoveAt(_tempHistory.Count - 1);
            }
        });
    }
}

public class TempRecord
{
    public string Time { get; set; } = string.Empty;
    public string Temperature { get; set; } = string.Empty;
}

```

实验结果

验证结果

平台	登录功能	输入验证	页面跳转	数据传递	温控功能
Windows	☒ 成功	☒ 成功	☒ 成功	☒ 成功	☒ 成功
Android	☐ 成功	☐ 成功	☐ 成功	☐ 成功	☐ 成功
iOS	☐ 成功	☐ 成功	☐ 成功	☐ 成功	☐ 成功

截图记录

MAUI登录页面

[插图位置：智能温控系统登录界面截图]

MAUI温控系统主页

[插图位置：智能温控系统主页截图]

Windows平台运行截图

[插图位置：Windows平台运行效果截图]

技术栈对比分析

对比项	MAUI (Windows)	Android	iOS
开发语言	C# 12.0	Kotlin	Swift
UI框架	XAML	XML/Compose	SwiftUI
页面导航	Navigation.PushAsync	Intent	NavigationLink
数据传递	构造函数参数	Intent.putExtra	@Binding
布局方式	StackLayout/Grid	线性布局/约束布局	Stack/Grid
生命周期	Page	Activity	ViewController
数据存储	本地字典/Preferences	Room	CoreData
热重载	☒ 支持	☒ 支持	☒ 支持

问题与解决

问题描述	解决方案	解决结果
Visual Studio 2019不支持MAUI开发	卸载2019版，安装Visual Studio 2022并勾选.NET MAUI工作负载	✅ 已解决
Android SDK路径包含中文导致识别失败	将SDK文件夹重命名为英文名称，更新Visual Studio中的SDK路径配置	✅ 已解决
NuGet包缓存路径包含中文导致编译错误	通过环境变量设置NUGET_PACKAGES为纯英文路径，清理NuGet缓存	✅ 已解决
TempRecord类属性非空警告	为Time和Temperature属性添加默认值 = string.Empty	✅ 已解决
Grid元素Spacing属性编译错误	使用Grid.ColumnSpacing和Grid.RowSpacing属性代替	✅ 已解决
页面导航失败	使用Navigation.PushAsync()方法进行页面跳转	✅ 已解决

实验总结

实验收获

- 完整掌握了MAUI项目的开发流程，从环境搭建、页面布局到功能实现的全流程
- 深入理解了XAML声明式UI与C#后台代码的协作模式，掌握了数据绑定与事件处理机制
- 实现了本地用户验证逻辑，掌握了用户名密码校验的实现方式
- 掌握了MAUI页面间导航与参数传递技术，理解了页面生命周期管理
- 培养了解决编译错误和环境问题的能力，学会了通过日志分析和官方文档排查问题

技术要点

- MAUI采用单一代码库支持多平台，通过.NET生态系统实现"一次编码，多端运行"
- XAML布局采用声明式设计，VerticalStackLayout、Grid等布局容器实现响应式界面
- C#异步编程模型async/await保证UI线程响应流畅，避免界面卡顿
- 使用Dictionary存储测试账号数据，实现本地用户名密码验证逻辑
- 设备控制采用互锁逻辑，避免空调与地暖同时开启的资源浪费问题
- 温度历史记录采用ObservableCollection实现数据驱动的UI自动更新

考核指标

考核项	评分标准	得分
功能完整性	登录功能完整，输入验证正确	25分

代码质量	代码结构清晰，注释完整	20分
页面跳转	Page跳转正常	15分
数据传递	构造函数传参正确	15分
用户验证	本地验证逻辑正确，测试账号可用	15分
实验报告	报告结构完整，内容详实	10分

教师评价

项目	评价
实验完成情况	
技术掌握程度	
创新点	
综合评价	
成绩	

教师签名： _____
日期： _____